

COMP101

Introduction to Programming

2025-26

Lecture-07

Numbers and Arithmetic

Three Numeric Data Types

- integer (e.g. 1, 10, 100, . . .)
 - ‘signed integers’
 - positive or negative whole numbers
 - no decimal point
- float (e.g. 1.0, 10.0, 100.0, . . .)
 - ‘floating point real numbers’
 - positive or negative real numbers
 - decimal point separates the integer and fractional parts
- complex (e.g. $a+bi$)
 - ‘complex numbers’
 - have a float part and an imaginary part

Don't need these
in COMP101



Integer Data Type

- Integer
 - Stored as 64 bit
 - max value = 9223372036854775807
 - min value = - 9223372036854775808
 - range = $-2^{63}-1$ to $(2^{63}-1)-1$
 - If we exceed 2^{64} Python keeps going until the number fills available RAM
 - Technically, 'int value is unlimited'
 - Boolean is a sub-type of integer (0 or 1)

NOTES:

- no longint, shortint or double in Python3.x
 - Python 2.x had 'int' at 32 bits and 'longint' at 64 bits

Float Data Type

- Float
 - Stored as 64-bit double-precision values
 - Single precision ('float precision') uses 32 bits
 - Double precision uses 64 bits
 - gives bigger numbers that are more accurate
 - max value = 1.7976931348623157e308
 - min value = 22.2250738585072014e-308

NOTES:

- 'e' or 'E' in the number is 'scientific notation'
 - 'e' or 'E' indicates the power of 10
 - $2.5e2 = 2.5 \times 10^2 = 250$

Complex Data Type

- Complex (Not needed in COMP101)
 - $\text{<real part>} + \text{<imaginary part>}$
 $a + bi$

a is float b is float i is $\sqrt{-1}$

NOTE:

Maths at this level is often better done in Python via an optional 'add-in' - NumPy

Numbers Operations

- Addition (+)
- Subtraction (-)
- Multiplication (*)
- Division (/, //, %)
 - true or integer, floor or DIV, Remainder or MOD ('Modulo')
- Exponent (**): e.g. $3^{**}2 = 9$
- Equality (==) not the same as assignment operator (=)

BODMAS is the same as elsewhere

- B – brackets
- O – orders (square root, exponent etc)
- D – division
- M – multiplication
- A – addition
- S – subtraction
- Operations of equal precedence calculate left-to-right
- If you get a wrong answer output, check the order of precedence and check brackets are in the right place

Addition

- Result outputs in field width big enough to display it fully
- Result is int or float depending on input

- $\text{int} + \text{int} = \text{int}$
 $10 + 3 = 13$

If we add integer to integer, we get an integer result

- $\text{int} + \text{float} = \text{float}$
 $10 + 2.5 = 12.5$
 $10 + 3.0 = 13.0$

If we add integer to float, we get a float result

#trailing zeroes suppressed

#user can enter 3, not 3.0 - it is still cast as float

- $\text{float} + \text{int} = \text{float}$
 $10.0 + 3 = 13.0$

If we add float to integer, we get a float result

- $\text{float} + \text{float} = \text{float}$
 $10.0 + 3.0 = 13.0$

If we add float to float, we get a float result

Subtraction, Multiplication, Exponentiation

- The result type is the same as addition in all cases
i.e. anything with a float in it gives a float as result
- int with int gives integer result
- int with float gives float result
- float with int gives float result
- float with float gives float result

You can always cast to change the result type

Add and Subtract Examples

- Addition (+) and Subtraction (–)

Addition and Subtraction

```
#addition
```

```
num1 = int(input("Enter first num: "))10  
num2 = float(input("Enter next num: "))5  
answer = (num1+num2)  
print ("Sum =",answer)  
Sum = 15.0
```

} If both integer then
output is integer:
If any number is
float then output is
float

```
#subtraction
```

```
num1 = int(input ("Enter first num: "))10  
num2 = float(input("Enter next num: "))5  
answer = (num1-num2)  
print ("Result =",answer)  
Result = 5.0
```

} If both integer then
output is integer:
If any number is
float then output is
float

Multiply and Exponent

- Multiplication (*) and Exponent (**)

Multiplication and Exponent

#multiplication

```
num1 = int(input("Enter first num: "))10
num2 = int(input("Enter next num: "))5
answer = (num1*num2)
print ("Product =",answer)
Product = 50
```

} If both integer then
output is integer:
If any number is
float then output is
float

#exponentiation

```
num1 = int(input("Enter first num: "))10
num2 = int(input("Enter next num: "))5
result = (num1**num2)
print ("Solution =",result)
Solution = 1000000
```

} If both integer then
output is integer:
If any number is
float then output is
float

Division overview

- Three types of division in Python
- `/` is True Division ('Integer Division')
- `//` is Floor Division ('DIV')
- `%` is Remainder Division ('Modulo' or 'MOD')

True Division (/) 'Integer Division'

- Result is always float (even if both numbers are integers)
- Result outputs as untruncated in field width 18 places inc d.pt
- int / int e.g. $10 / 3 = 3.3333333333333335$ **#18dp**
- int / float e.g. $10 / 2.5 = 4.0$ **#trailing zeroes are suppressed**
 $10 / 3.0 = 3.3333333333333335$
- float / int e.g. $10.0 / 3 = 3.3333333333333335$
- float / float e.g. $10.0 / 3.0 = 3.3333333333333335$

Floor Division (//) 'DIV'

- Gives the 'whole' part truncated (rounded down)
- Result is integer (truncated) when both integer
- Result is float (truncated) when any number is a float
- `int // int` e.g. `10//3 = 3` *#10 divided by 3 is 3 remainder 1*
- `int // float` e.g. `10 // 2.5 = 4.0` *#trailing zeroes suppressed*
 `10 // 3.0 = 3.0`
- `float // int` e.g. `10.0 // 3 = 3.0`
- `float // float` e.g. `10.0 // 3.0 = 3.0`

Remainder Division ('MOD')

- Gives the remainder (rounded down)
- Gives integer when both numbers are integer
- Gives truncated float when any number is a float
- `int % int` e.g. `10 % 3 = 1` #10 divided by 3 is 3 remainder 1
- `int % float` e.g. `10 % 2.5 = 0.0`
 `10 % 3.0 = 1.0`
- `float % int` e.g. `10.0 % 3 = 1.0`
- `float % float` e.g., `10.0 % 3.0 = 1.0`

Division examples

- True division (/), Floor division (//)

Division giving integer and float

#True division - output always untruncated float

```
num1 = int(input("Enter first num: "))10
```

```
num2 = int(input("Enter next num: "))3
```

```
result = (num1/num2)
```

```
print ("Solution =",result)
```

```
Solution = 3.3333333333333335
```

Field width is 18
including d.pt

#Floor division - output is integer if both integer,
else truncated float

```
result = (num1//num2)
```

```
print ("Solution =",result)
```

```
Solution = 3
```

Division examples

- Division Remainder (%) – outputs the remainder

Modulo Division (%)

```
#integer modulo  
c = 10  
d = 3  
print(c%d)  
1
```

Can be used as test for divisibility:
if $x\%y = 0$ then x is divisible by y

```
#float(decimal) modulo:  
e = 7.5  
f = 2.4  
print(e%f)  
0.30000000000000027 #untruncated
```

Floating point arithmetic

Issues and limitations

- $10 / 3 = 3.333333333333333335$
 - Result outputs as untruncated result in field width 18 places inc d.pt
 - Where did the last digit (5) come from?
 - Shouldn't it be 3.3333333333333333?
- In any language, it is recognised floats have an inaccuracy
 - Caused by the binary representation of the float and how it is stored by the hardware
- In binary format, the above answer translates as:
3.33333333333333334813630699500208720564842224121094
- Rounding to 16 dp gives 3.3333333333333333
- Not critical unless this degree of accuracy is required (exceptional)